

- 1] Implement simple and multiple linear regression using Numpy and using Scikit-learn. Plot regression lines and residuals. Compute RMSE and R2 values.

- 1] Import Libraries

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

- 2] Data Collection

```
# Create the dataset
data = {
    "Study_Hours": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Attendance": [60, 85, 70, 95, 65, 80, 90, 75, 88, 72],
    "Marks": [32, 42, 47, 58, 61, 67, 76, 79, 88, 91] }

# Convert dict into DataFrame
df = pd.DataFrame(data)
print(df)
```

- 3] EDA And Preprocessing

```
print('First Five Rows:')
df.head(3)
```

```
df.tail()
```

```
print('Shape of Dataset:')
df.shape
```

```
print('Information:')
df.info()
```

```
print('Statistical Analysis:\n', df.describe())
```

```
print('Check for null values:')
df.isnull().sum()
```

```
df=df.dropna()
df
```

```
# Duplicate Values
df.duplicated().sum()
```

```
df=df.drop_duplicates()
df
```

- Data Visualization

```
df_corr=df.corr()  
df_corr
```

```
# Correlation Heatmap  
import seaborn as sns  
  
plt.figure(figsize= (6, 4))  
sns.heatmap(df_corr, annot=True, cmap='Blues')  
plt.title('Correlation Heatmap')
```

✓ A] Simple Linear Regression by Using Sckit-learn

```
# Feature Selection  
x=df[['Study_Hours']] #Independent Variable  
y=df['Marks']         #Dependent Variable
```

```
x
```

✓ 4] Model Building

```
# Split dataset into training and testing set  
from sklearn.model_selection import train_test_split  
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=42)
```

```
x_test
```

```
# Import Model  
from sklearn.linear_model import LinearRegression  
model = LinearRegression()  
  
# Train the model  
model.fit(x_train, y_train)
```

```
#Predict the values  
y_pred=model.predict(x_test)  
y_pred
```

```
# Print Equation  
print('Intercept:', model.intercept_)  
print('Slope:', model.coef_[0])
```

SLR Equation

- Marks= (6.48* Study_Hours) + 28.27

✓ 5]Model Evaluation

```
#Evaluation  
from sklearn.metrics import root_mean_squared_error, r2_score, mean_squared_error  
rmse = np.sqrt(mean_squared_error(y_test, y_pred))  
r2= r2_score(y_test, y_pred)  
  
print('RSME:', round(rmse, 3))  
print('R2 Score:', round(r2, 2))
```

```
# Regression Line
plt.figure(figsize=(7, 5))
plt.scatter(x, y, color='blue', label='Actual Data')
plt.plot(x_test, y_pred,
         color='red', label='Regression Line')

plt.xlabel('Study Hours')
plt.ylabel('Marks')
plt.title('Simple Linear Regression')
plt.legend()
plt.show()
```

✓ Residual Plot

```
residuals = y_test - y_pred # Calculate Residual

plt.figure(figsize=(7, 5))
plt.scatter(y_pred, residuals, color='green')
plt.axhline(y=0, color='red', linestyle="--")
plt.xlabel('Predicted Marks')
plt.ylabel('Residuals')
plt.title('Residual Plot')
plt.show()
```

✓ B] Simple Linear Regression by using Numpy

```
x = df["Study_Hours"].values # Independent Variable (Features)

y = df["Marks"].values # Dependent Variable (Target)
```

```
x
```

```
# Calculate mean
x_mean = np.mean(x)
y_mean = np.mean(y)
```

```
x_mean
```

```
# Calculate slope
m = np.sum((x-x_mean)*(y-y_mean))/np.sum((x-x_mean)**2)

# Calculate intercept
b = y_mean - m*x_mean

print("Slope =", m)
print("Intercept =", b)
```

```
# Prediction
y_pred_numpy = m*x + b
y_pred_numpy
```

✓ c] Multiple Linear Regression

```
# Feature Selection
x = df[['Study_Hours', 'Attendance']] # Independent Variables
y = df['Marks'] # Dependent Variable

# Split Dataset
```

```
x_train, x_test, y_train, y_test =train_test_split(x, y, test_size=0.3,  
                                                  random_state=42)
```

```
x_train
```

✓ Model Building

```
model=LinearRegression()  
model.fit(x_train, y_train)
```

```
# Prediction  
y_pred =model.predict(x_test)  
y_pred
```

```
# Model Parameters  
print('Intercept:', model.intercept_)  
print("Coefficients",)  
coeff=pd.DataFrame({  
    'Feature': x.columns, 'Coefficient': model.coef_ })  
  
print(coeff)
```

✓ Model Evalution

```
# Evaluation Metrics  
from sklearn.metrics import mean_absolute_error  
  
print('Metrics Values of MLR:')  
rmse = np.sqrt(mean_squared_error(y_test, y_pred))  
r2 = r2_score(y_test, y_pred)  
mae=mean_absolute_error(y_test, y_pred)  
  
print('MAE:', mae)  
print('RMSE:', round(rmse, 2))  
print('R2 Score:', round(r2, 2))
```

```
# Actual vs Predicted Plot  
plt.figure(figsize=(7, 5))  
plt.scatter(y_test, y_pred, color='Purple')  
  
# Perfect Prediction Line  
plt.plot([y.min(), y.max()],  
         [y_pred.min(), y_pred.max()], color='red')  
  
plt.xlabel('Actual Marks')  
plt.ylabel("predicted Marks")  
plt.title('Actual vs Predicted (MLR)')  
plt.show()
```

✓ Residual Plot

```
residuals = y_test - y_pred #calculate residual  
  
plt.scatter(y_pred, residuals, color='Orange')  
plt.axhline(y=0, color='red', linestyle='--')  
plt.xlabel('Predicted Marks')  
plt.ylabel('Residuals')  
plt.title('Residual plot(MLR)')  
plt.show()
```